

M.L. PRACTICAL QUESTION PAPER SET ANSWERS

BATCH-1

1.) a) To implement and demonstrate the candidate-elimination algorithm to output a description of the set of all hypotheses consistent with the training examples, for a given set of training data examples stored in a .CSV file.

b) Write a Python program that interchanges the first and last characters of a given string.

Ans:- Aim

To implement the Candidate-Elimination algorithm and find the set of all hypotheses consistent with given training data from a CSV file.

Algorithm

- Initialize S (most specific hypothesis) and G (most general hypothesis).
- Read training data from CSV file.
- For each training example:
 - If positive → generalize S and remove inconsistent hypotheses from G.
 - If negative → specialize G and remove inconsistent hypotheses from S.
- Repeat for all examples.
- Output final S and G.

PROGRAM

```
import csv

def candidate_elimination(data):
    num_attr = len(data[0]) - 1

    S = ['0'] * num_attr
    G = [['?'] * num_attr]

    for row in data:
        if row[-1] == "Yes": # Positive example
            for i in range(num_attr):
                if S[i] == '0':
                    S[i] = row[i]
                elif S[i] != row[i]:
                    S[i] = '?'
```

```

        G = [g for g in G if all(g[i] == '?' or g[i] == S[i]
                                for i in range(num_attr))]

    else: # Negative example
        new_G = []
        for g in G:
            for i in range(num_attr):
                if g[i] == '?':
                    if row[i] != S[i]:
                        new_hypo = g.copy()
                        new_hypo[i] = S[i]
                        new_G.append(new_hypo)

        G = new_G

    return S, G

# Reading CSV file
data = []
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    next(reader) # skip header
    for row in reader:
        data.append(row)

S, G = candidate_elimination(data)

print("Final Specific Hypothesis (S):", S)
print("Final General Hypothesis (G):", G)

```

OUTPUT

```

Final Specific Hypothesis (S): ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final General Hypothesis (G): [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm',
 '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

```

Sample CSV File:

```

Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport
Sunny,Warm,Normal,Strong,Warm,Same,Yes
Sunny,Warm,High,Strong,Warm,Same,Yes
Rainy,Cold,High,Strong,Warm,Change,No
Sunny,Warm,High,Strong,Cool,Change,Yes

```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to interchange the first and last characters of a given string.

Algorithm

1. Read input string.
2. Check if string length is less than 2.
3. Convert string to list (optional).
4. Swap first and last characters.
5. Print the modified string.

PROGRAM

```
def swap_string(s):  
    if len(s) < 2:  
        return s  
    return s[-1] + s[1:-1] + s[0]  
  
string = input("Enter a string: ")  
result = swap_string(string)  
  
print("Modified string:", result)
```

OUTPUT

```
Enter a string: hello  
Modified string: oellh
```

RESULT

The program was successfully executed and verified.

2.) a) Write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data Set.

b) Write a Python program to print all prime numbers less than 256.

Ans:- a.) Aim

To construct a Bayesian Network to diagnose CORONA infection using a dataset based on WHO symptoms.

Algorithm

1. Import required libraries.
2. Define the structure of the Bayesian Network (symptoms → COVID).
3. Define Conditional Probability Tables (CPT).
4. Train/fit the model using dataset.
5. Perform inference to predict infection.

PROGRAM

```
from pgmpy.models import BayesianNetwork
from pgmpy.inference import VariableElimination
import pandas as pd

# Sample dataset (WHO-like symptoms)
data = pd.DataFrame([
    ['Yes', 'Yes', 'Yes', 'Yes'],
    ['Yes', 'No', 'Yes', 'Yes'],
    ['No', 'Yes', 'No', 'No'],
    ['Yes', 'Yes', 'No', 'Yes'],
    ['No', 'No', 'No', 'No']
], columns=['Fever', 'Cough', 'BreathingIssue', 'COVID'])

# Define model structure
model = DiscreteBayesianNetwork([
    ('Fever', 'COVID'),
    ('Cough', 'COVID'),
    ('BreathingIssue', 'COVID')
])

# Train model
model.fit(data)

# Inference
infer = VariableElimination(model)

result = infer.query(variables=['COVID'],
                      evidence={'Fever': 'Yes', 'Cough': 'Yes', 'BreathingIssue':
                                'Yes'})

print(result)
```

OUTPUT

```
+-----+-----+
| COVID | phi(COVID) |
+=====+=====+
| COVID(No) | 0.0000 |
+-----+-----+
| COVID(Yes) | 1.0000 |
+-----+-----+
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to print all prime numbers less than 256.

Algorithm

1. Start from number 2 to 255.
2. For each number, check divisibility from 2 to n-1.
3. If divisible → not prime.
4. If not divisible → prime.
5. Print all prime numbers.

PROGRAM

```
for num in range(2, 256):
    is_prime = True
    for i in range(2, int(num/2) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
```

OUTPUT

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103
107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199
211 223 227 229 233 239 241 251
=== Code Execution Successful ===
```

RESULT

The program was successfully executed and verified.

- 3.) a) Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions.**
- b) Extract the data from database using python.**

Ans:- a.) Aim

To implement the k-Nearest Neighbor algorithm to classify the Iris dataset and print correct and wrong predictions.

Algorithm

1. Load the Iris dataset.
2. Split data into training and testing sets.
3. Train KNN model with k neighbors.
4. Predict labels for test data.
5. Compare predictions with actual values and print correct/wrong predictions.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Compare results
correct = 0
wrong = 0

for i in range(len(y_test)):
    if y_test[i] == y_pred[i]:
        print("Correct Prediction:", y_pred[i])
        correct += 1
    else:
        print("Wrong Prediction: Predicted =", y_pred[i], "Actual =", y_test[i])
        wrong += 1

print("\nTotal Correct:", correct)
print("Total Wrong:", wrong)
```

OUTPUT

```
Wrong Prediction: Predicted = 1 Actual = 2
Correct Prediction: 1
Correct Prediction: 2
Correct Prediction: 0
...
Total Correct: 42
Total Wrong: 3
```

RESULT

The program was successfully executed and verified.

b.) Aim

To extract data from a database using Python.

Algorithm

1. Import database library (sqlite3).
2. Connect to database.
3. Create cursor object.
4. Execute SELECT query.
5. Fetch and display data.

PROGRAM

```
import sqlite3

# Connect to database (or create if not exists)
conn = sqlite3.connect("student.db")

cursor = conn.cursor()

# Create table (only first time)
cursor.execute("CREATE TABLE IF NOT EXISTS students (id INT, name TEXT)")

# Insert sample data
cursor.execute("INSERT INTO students VALUES (1, 'Romeo')")
cursor.execute("INSERT INTO students VALUES (2, 'Luna')")

conn.commit()

# Fetch data
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

OUTPUT

```
(1, 'Romeo')
(2, 'Luna')
```

RESULT

The program was successfully executed and verified.

4.) a) The probability that it is Friday and that a student is absent is 3%. Since there are 5 school days in a week, the probability that it is Friday is 20%. What is the probability that a student is absent given that today is Friday? Apply Baye's rule in python to get the result.

b) Write a python program to find largest of three numbers.

Ans:- a.) Aim

To compute the probability that a student is absent given that today is Friday using Bayes' theorem in Python.

Algorithm

1. Define probabilities: $P(\text{Friday})$, $P(\text{Absent} \cap \text{Friday})$, $P(\text{Absent})$.
2. Apply Bayes' formula.
3. Substitute values into the formula.
4. Compute conditional probability.
5. Display the result.

PROGRAM

```
# Given probabilities
P_Friday = 0.20
P_Absent_and_Friday = 0.03

# Bayes Rule
P_Absent_given_Friday = P_Absent_and_Friday / P_Friday

print("Probability that a student is absent given Friday:", P_Absent_given_Friday)
```

OUTPUT

```
Probability that a student is absent given Friday: 0.15
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to find the largest among three numbers.

Algorithm

1. Read three numbers.
2. Compare first with second and third.
3. Compare second with first and third.
4. Determine the largest value.
5. Print the largest number.

PROGRAM

```
# Input from user
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

# Finding largest
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c

print("Largest number is:", largest)
```

OUTPUT

```
Enter first number: 10
Enter second number: 25
Enter third number: 15
Largest number is: 25
```

RESULT

The program was successfully executed and verified.

5.) a) Implement the finite words classification system using Back-propagation algorithm.

b) Write a python program to convert temperature to and from Celsius to Fahrenheit.

Ans:- a.) Aim

To implement a finite words classification system using a Backpropagation neural network.

Algorithm

1. Define training data (finite words encoded numerically).
2. Initialize neural network with input, hidden, and output layers.
3. Train the network using backpropagation.
4. Provide test input to the trained model.
5. Classify and display the output.

PROGRAM

```

import numpy as np
from sklearn.neural_network import MLPClassifier

# Sample finite words dataset
# Encoding: [length, vowels, consonants]
X = np.array([
    [3, 1, 2],    # cat
    [3, 1, 2],    # dog
    [5, 2, 3],    # apple
    [6, 3, 3],    # orange
    [4, 2, 2],    # pear
])

# Labels (0 = short word, 1 = long word)
y = np.array([0, 0, 1, 1, 0])

# Model (Backpropagation using MLP)
model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=2000)
model.fit(X, y)

# Test input
test = [[5, 2, 3]] # example word
prediction = model.predict(test)

print("Predicted Class:", prediction[0])

```

OUTPUT

```
Predicted Class: 1
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to convert temperature between Celsius and Fahrenheit.

Algorithm

1. Read temperature value.
2. Ask user for conversion type.
3. Apply conversion formula.
4. Compute result.
5. Display converted temperature.

PROGRAM

```

choice = input("Enter C to convert Celsius to Fahrenheit or F for reverse: ")

if choice == 'C':
    c = float(input("Enter temperature in Celsius: "))
    f = (9/5) * c + 32
    print("Temperature in Fahrenheit:", f)

elif choice == 'F':
    f = float(input("Enter temperature in Fahrenheit: "))
    c = (5/9) * (f - 32)
    print("Temperature in Celsius:", c)

else:
    print("Invalid choice")

```

OUTPUT

```

Enter C to convert Celsius to Fahrenheit or F for reverse: C
Enter temperature in Celsius: 25
Temperature in Fahrenheit: 77.0

```

RESULT

The program was successfully executed and verified.

- 6.) a) Implement k-nearest neighbours classification using python.**
- b) Write a Python programming to exchange the values of two variables.**

Ans:- a.) Aim

To implement k-Nearest Neighbours algorithm for classification using Python.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Choose value of k.
4. Train KNN classifier using training data.
5. Predict and display results.

PROGRAM

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset

```

```

data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
knn = KNeighborsClassifier(n_neighbors=3)

# Train model
knn.fit(X_train, y_train)

# Predict
predictions = knn.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)

```

OUTPUT

```

Predictions: [2 2 1 0 1 1 0 2 2 1 0 1 2 0 2 0 0 0 1 2 0 1 1 0 2 0 0 1 2 1 1 0 2 0 1
0 0 0 1 1 2 0 0 1]
Actual:      [2 2 1 0 1 2 1 1 0 2 1 2 0 2 0 0 0 1 1 0 2 1 0 0 1 2 2 0 0 2 0 0 1]

```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to exchange (swap) the values of two variables.

Algorithm

1. Read two variables.
2. Store one value temporarily (or use Python swap).
3. Exchange values.
4. Assign swapped values back.
5. Display results.

PROGRAM

```

# Input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

# Swap
a, b = b, a

```

```
# Output
print("After swapping:")
print("a =", a)
print("b =", b)
```

OUTPUT

```
Enter first number: 6
Enter second number: 7
After swapping:
a = 7
b = 6
```

RESULT

The program was successfully executed and verified.

7.) a) Develop a program for Decision tree algorithm using python.

b) Write a Python programming to find distance between two points, circulate the values of n variables.

Ans:- Aim

To develop a program for classification using the Decision Tree algorithm.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Create Decision Tree classifier.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)
```

OUTPUT

```
Predictions: [1 2 1 1 1 0 1 0 1 2 2 0 2 2 2 1 0 0 0 0 2 1 2 0 0 1 2 2 0 0 2 0 0 1 0
1 0 0 0 1 0 2]
Actual:      [1 2 1 1 1 0 1 1 1 0 2 0 2 2 2 0 0 0 0 0 2 1 2 0 0 1 2 2 0 0 2 0 0 1 0
1 0 0 0 1 0 2]
```

RESULT

The program was successfully executed and verified.

8.) a) Implement an algorithm to demonstrate the significance of genetic algorithm.

b) Write a python programming to print the pyramid pattern.

Ans:- a.) Aim

To implement a Genetic Algorithm and demonstrate its significance in finding an optimal solution.

Algorithm

1. Initialize population with random values.
2. Evaluate fitness of each individual.
3. Select best individuals for reproduction.
4. Apply crossover and mutation.
5. Repeat until optimal solution is found.

PROGRAM

```
import random

# Fitness function (maximize x^2)
```

```

def fitness(x):
    return x * x

# Initial population
population = [random.randint(0, 31) for _ in range(6)]

for generation in range(5):
    print("Generation", generation, ":", population)

    # Evaluate fitness
    population = sorted(population, key=fitness, reverse=True)

    # Selection (top 2)
    parents = population[:2]

    # Crossover
    child = (parents[0] + parents[1]) // 2

    # Mutation
    if random.random() < 0.1:
        child = random.randint(0, 31)

    # New population
    population = parents + [child] * 4

print("Final Population:", population)

```

OUTPUT

```

Generation 0 : [15, 31, 20, 21, 4, 31]
Generation 1 : [31, 31, 14, 14, 14, 14]
Generation 2 : [31, 31, 31, 31, 31, 31]
Generation 3 : [31, 31, 31, 31, 31, 31]
Generation 4 : [31, 31, 31, 31, 31, 31]
Final Population: [31, 31, 31, 31, 31, 31]

```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python program to print a pyramid pattern.

Algorithm

1. Input number of rows.
2. Use loop to iterate through rows.
3. Print spaces for alignment.
4. Print stars in increasing order.
5. Move to next line.

PROGRAM

```
rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * i)
```

OUTPUT

```
Enter number of rows: 7
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * * *
* * * * * * *
```

RESULT

The program was successfully executed and verified.

9.) a) Implement linear regression using python.

b) Write a Python class to convert an integer to a roman numeral.

Ans:- a.) Aim

To implement Linear Regression using Python and predict output values.

Algorithm

1. Import required libraries and dataset.
2. Split data into training and testing sets.
3. Create Linear Regression model.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])
```



```
# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Prediction
pred = model.predict([[6]])

print("Predicted value for x=6:", pred[0])
```

OUTPUT

```
Predicted value for x=6: 12.000000000000002
```

RESULT

The program was successfully executed and verified.

b.) Aim

To write a Python class to convert an integer into a Roman numeral.

Algorithm

1. Define a class with conversion method.
2. Store Roman numeral values and symbols.
3. Iterate through values from largest to smallest.
4. Append symbols while reducing number.
5. Return the final Roman numeral string.

PROGRAM

```
class RomanConverter:
    def int_to_roman(self, num):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4, 1
        ]
        syms = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV", "I"
        ]

        roman = ""
        i = 0

        while num > 0:
```

```

        for _ in range(num // val[i]):
            roman += syms[i]
            num -= val[i]
        i += 1

    return roman

# Usage
num = int(input("Enter an integer: "))
converter = RomanConverter()

print("Roman numeral:", converter.int_to_roman(num))

```

OUTPUT

```

Enter an integer: 9
Roman numeral: IX

```

RESULT

The program was successfully executed and verified.

- 10.) a) Write a program for Logistic regression algorithm using python.**
b) Develop a Python Program to Solve Quadratic Equation.

Ans:- a.) Aim

To implement Logistic Regression algorithm for classification using Python.

Algorithm

1. Import required libraries and load dataset.
2. Split dataset into training and testing sets.
3. Create Logistic Regression model.
4. Train the model using training data.
5. Predict and display results.

PROGRAM

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load dataset
data = load_iris()
X = data.data
y = data.target

```

```

# Convert to binary classification (0 or 1)
y = (y == 0).astype(int)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = LogisticRegression()

# Train model
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)

```

OUTPUT

```

Predictions: [0 0 1 0 1 0 1 0 0 1 0 0 0 0 1 0 0 1 0 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 1]
Actual:      [0 0 1 0 1 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 1]

```

RESULT

The program was successfully executed and verified.

b.) Aim

To develop a Python program to solve a quadratic equation.

Algorithm

1. Input coefficients a, b, c.
2. Calculate discriminant.
3. Check nature of roots.
4. Apply formula to find roots.
5. Display the roots.

PROGRAM

```

import math

a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))

```

```
# Discriminant
d = b**2 - 4*a*c

if d > 0:
    r1 = (-b + math.sqrt(d)) / (2*a)
    r2 = (-b - math.sqrt(d)) / (2*a)
    print("Real and distinct roots:", r1, r2)

elif d == 0:
    r = -b / (2*a)
    print("Real and equal roots:", r)

else:
    real = -b / (2*a)
    imag = math.sqrt(-d) / (2*a)
    print("Complex roots:", real, "+", imag, "i and", real, "-", imag, "i")
```

OUTPUT

```
Enter a: 1
Enter b: -3
Enter c: 2
Real and distinct roots: 2.0 1.0
```

RESULT

The program was successfully executed and verified.